

Praktikum 9: Mass-Storage Structure

Informasi Praktikum

- **Mata Kuliah:** Sistem Operasi
 - **Sistem Operasi:** Linux Ubuntu
 - **Bahasa Pemrograman:** C
 - **Dosen Pengampu:** Triawan Adi Cahyanto, M.Kom
 - **Asisten Dosen:** Atidhira Habibillah dan Taqiyyuddin
-

Pendahuluan

Tujuan Praktikum

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan dapat:

1. Memahami struktur dan hierarki penyimpanan massal
2. Mengimplementasikan dan menganalisis algoritma penjadwalan disk
3. Mengelola perangkat penyimpanan di Linux

Persiapan Praktikum

Software yang Dibutuhkan:

- Sistem Operasi: Ubuntu 20.04 atau lebih baru
- Compiler: GCC
- Tools: vim/nano, make, fdisk, parted, mdadm

Instalasi Tools:

```
sudo apt update
sudo apt install build-essential vim nano parted hdparm smartmontools
mdadm
```

Topik 1: Overview of Mass-Storage Structure

Tujuan

Memahami hierarki penyimpanan dan karakteristik perangkat storage di sistem Linux.

Langkah-Langkah Praktikum

1.1 Melihat Informasi Storage Device

Langkah 1: Buka terminal dan jalankan perintah untuk melihat semua perangkat penyimpanan

```
lsblk
```

Output yang diharapkan:

NAME	MAJ:MIN	RM	SIZE	RO	TYPE	MOUNTPOINT
sda	8:0	0	100G	0	disk	
_sda1	8:1	0	99G	0	part	/
_sda2	8:2	0	1G	0	part	[SWAP]

Penjelasan:

- **NAME:** nama perangkat
- **SIZE:** ukuran storage
- **TYPE:** tipe (disk, partition)
- **MOUNTPOINT:** lokasi mount

Langkah 2: Melihat detail informasi disk

```
sudo fdisk -l
```

Langkah 3: Melihat informasi partisi dengan detail

```
df -h
```

Langkah 4: Melihat hierarki penyimpanan secara visual

```
lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINT,MODEL
```

1.2 Membuat Program C untuk Analisis Storage Hierarchy

Langkah 1: Buat direktori kerja

```
mkdir ~/praktikum_storage  
cd ~/praktikum_storage
```

Langkah 2: Buat file `storage_info.c`

```
nano storage_info.c
```

Langkah 3: Ketik kode berikut:

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/statvfs.h>
#include <unistd.h>

void print_storage_info(const char *path) {
    struct statvfs stat;

    if (statvfs(path, &stat) != 0) {
        perror("statvfs");
        return;
    }

    unsigned long total_space = stat.f_blocks * stat.f_frsize;
    unsigned long free_space = stat.f_bfree * stat.f_frsize;
    unsigned long available_space = stat.f_bavail * stat.f_frsize;
    unsigned long used_space = total_space - free_space;

    printf("=== Storage Information for: %s ===\n", path);
    printf("Total Space      : %.2f GB\n", total_space / (1024.0 * 1024.0 *
1024.0));
    printf("Used Space       : %.2f GB\n", used_space / (1024.0 * 1024.0 *
1024.0));
    printf("Free Space      : %.2f GB\n", free_space / (1024.0 * 1024.0 *
1024.0));
    printf("Available Space : %.2f GB\n", available_space / (1024.0 *
1024.0 * 1024.0));
    printf("Block Size      : %lu bytes\n", stat.f_bsize);
    printf("Total Inodes    : %lu\n", stat.f_files);
    printf("Free Inodes     : %lu\n", stat.f_ffree);
    printf("=====\n\n");
}

int main() {
    printf("\n*** HIERARKI PENYIMPANAN MASSAL ***\n\n");

    print_storage_info("/");

    // Cek apakah ada mount point lain
    if (access("/home", F_OK) == 0) {
        print_storage_info("/home");
    }
}
```

```
    return 0;  
}
```

Langkah 4: Compile program

```
gcc -o storage_info storage_info.c
```

Langkah 5: Jalankan program

```
./storage_info
```

Topik 2: HDD Scheduling Algorithms

Tujuan

Mengimplementasikan dan membandingkan algoritma penjadwalan disk (FCFS, SSTF, SCAN, C-SCAN, LOOK, C-LOOK).

Langkah-Langkah Praktikum

2.1 Simulasi Algoritma FCFS (First-Come-First-Served)

Langkah 1: Buat file `disk_scheduling.c`

```
nano disk_scheduling.c
```

Langkah 2: Ketik kode lengkap berikut:

```
#include <stdio.h>  
#include <stdlib.h>  
#include <math.h>  
  
#define MAX_REQUESTS 100  
  
// Fungsi untuk menghitung total head movement  
int calculate_total_movement(int requests[], int n, int initial_head) {  
    int total = 0;  
    int current = initial_head;  
  
    for (int i = 0; i < n; i++) {  
        total += abs(requests[i] - current);  
        current = requests[i];  
    }  
}
```

```
        return total;
    }

// Algoritma FCFS
void fcfs(int requests[], int n, int initial_head) {
    printf("\n=== FCFS (First-Come-First-Served) ===\n");
    printf("Urutan akses: %d", initial_head);

    int current = initial_head;
    int total = 0;

    for (int i = 0; i < n; i++) {
        printf(" -> %d", requests[i]);
        total += abs(requests[i] - current);
        current = requests[i];
    }

    printf("\nTotal head movement: %d cylinders\n", total);
}

// Algoritma SSTF (Shortest Seek Time First)
void sstf(int requests[], int n, int initial_head) {
    printf("\n=== SSTF (Shortest Seek Time First) ===\n");

    int visited[MAX_REQUESTS] = {0};
    int current = initial_head;
    int total = 0;
    int count = 0;

    printf("Urutan akses: %d", initial_head);

    while (count < n) {
        int min_distance = 99999;
        int min_index = -1;

        // Cari request terdekat yang belum dikunjungi
        for (int i = 0; i < n; i++) {
            if (!visited[i]) {
                int distance = abs(requests[i] - current);
                if (distance < min_distance) {
                    min_distance = distance;
                    min_index = i;
                }
            }
        }

        if (min_index != -1) {
            visited[min_index] = 1;
            total += min_distance;
            current = requests[min_index];
            printf(" -> %d", current);
            count++;
        }
    }
}
```

```

    }

    printf("\nTotal head movement: %d cylinders\n", total);
}

// Fungsi untuk sorting (bubble sort)
void sort_requests(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

// Algoritma SCAN
void scan(int requests[], int n, int initial_head, int disk_size, int
direction) {
    printf("\n=== SCAN (Elevator Algorithm) ===\n");
    printf("Direction: %s\n", direction == 1 ? "Right (increasing)" :
"Left (decreasing)");

    int sorted[MAX_REQUESTS];
    for (int i = 0; i < n; i++) {
        sorted[i] = requests[i];
    }
    sort_requests(sorted, n);

    int current = initial_head;
    int total = 0;

    printf("Urutan akses: %d", initial_head);

    if (direction == 1) { // Bergerak ke kanan dulu
        // Layani request di sebelah kanan
        for (int i = 0; i < n; i++) {
            if (sorted[i] >= current) {
                total += abs(sorted[i] - current);
                current = sorted[i];
                printf(" -> %d", current);
            }
        }

        // Pergi ke ujung kanan
        if (current != disk_size - 1) {
            total += abs((disk_size - 1) - current);
            current = disk_size - 1;
            printf(" -> %d", current);
        }

        // Balik dan layani request di sebelah kiri

```

```

        for (int i = n - 1; i >= 0; i--) {
            if (sorted[i] < initial_head) {
                total += abs(current - sorted[i]);
                current = sorted[i];
                printf(" -> %d", current);
            }
        }
    } else { // Bergerak ke kiri dulu
        // Layani request di sebelah kiri
        for (int i = n - 1; i >= 0; i--) {
            if (sorted[i] <= current) {
                total += abs(current - sorted[i]);
                current = sorted[i];
                printf(" -> %d", current);
            }
        }

        // Pergi ke ujung kiri
        if (current != 0) {
            total += current;
            current = 0;
            printf(" -> %d", current);
        }

        // Balik dan layani request di sebelah kanan
        for (int i = 0; i < n; i++) {
            if (sorted[i] > initial_head) {
                total += abs(sorted[i] - current);
                current = sorted[i];
                printf(" -> %d", current);
            }
        }
    }

    printf("\nTotal head movement: %d cylinders\n", total);
}

// Algoritma C-SCAN
void c_scan(int requests[], int n, int initial_head, int disk_size) {
    printf("\n=== C-SCAN (Circular SCAN) ===\n");

    int sorted[MAX_REQUESTS];
    for (int i = 0; i < n; i++) {
        sorted[i] = requests[i];
    }
    sort_requests(sorted, n);

    int current = initial_head;
    int total = 0;

    printf("Urutan akses: %d", initial_head);

    // Layani request di sebelah kanan
    for (int i = 0; i < n; i++) {

```

```

        if (sorted[i] >= current) {
            total += abs(sorted[i] - current);
            current = sorted[i];
            printf(" -> %d", current);
        }
    }

    // Pergi ke ujung kanan
    if (current != disk_size - 1) {
        total += abs((disk_size - 1) - current);
        current = disk_size - 1;
        printf(" -> %d", current);
    }

    // Langsung ke ujung kiri (tanpa melayani)
    total += (disk_size - 1); // Kembali ke 0
    current = 0;
    printf(" -> %d (jump)", current);

    // Layani request di sebelah kiri dari posisi awal
    for (int i = 0; i < n; i++) {
        if (sorted[i] < initial_head) {
            total += abs(sorted[i] - current);
            current = sorted[i];
            printf(" -> %d", current);
        }
    }

    printf("\nTotal head movement: %d cylinders\n", total);
}

// Algoritma LOOK
void look(int requests[], int n, int initial_head, int direction) {
    printf("\n=== LOOK ===\n");
    printf("Direction: %s\n", direction == 1 ? "Right (increasing)" :
"Left (decreasing)");

    int sorted[MAX_REQUESTS];
    for (int i = 0; i < n; i++) {
        sorted[i] = requests[i];
    }
    sort_requests(sorted, n);

    int current = initial_head;
    int total = 0;

    printf("Urutan akses: %d", initial_head);

    if (direction == 1) { // Bergerak ke kanan dulu
        // Layani request di sebelah kanan
        for (int i = 0; i < n; i++) {
            if (sorted[i] >= current) {
                total += abs(sorted[i] - current);
                current = sorted[i];
            }
        }
    }
}

```

```

        printf(" -> %d", current);
    }
}

// Balik dan layani request di sebelah kiri
for (int i = n - 1; i >= 0; i--) {
    if (sorted[i] < initial_head) {
        total += abs(current - sorted[i]);
        current = sorted[i];
        printf(" -> %d", current);
    }
}
} else { // Bergerak ke kiri dulu
    // Layani request di sebelah kiri
    for (int i = n - 1; i >= 0; i--) {
        if (sorted[i] <= current) {
            total += abs(current - sorted[i]);
            current = sorted[i];
            printf(" -> %d", current);
        }
    }
}

// Balik dan layani request di sebelah kanan
for (int i = 0; i < n; i++) {
    if (sorted[i] > initial_head) {
        total += abs(sorted[i] - current);
        current = sorted[i];
        printf(" -> %d", current);
    }
}
}

printf("\nTotal head movement: %d cylinders\n", total);
}

// Algoritma C-LOOK
void c_look(int requests[], int n, int initial_head) {
    printf("\n=== C-LOOK (Circular LOOK) ===\n");

    int sorted[MAX_REQUESTS];
    for (int i = 0; i < n; i++) {
        sorted[i] = requests[i];
    }
    sort_requests(sorted, n);

    int current = initial_head;
    int total = 0;

    printf("Urutan akses: %d", initial_head);

    // Layani request di sebelah kanan
    for (int i = 0; i < n; i++) {
        if (sorted[i] >= current) {
            total += abs(sorted[i] - current);

```

```

        current = sorted[i];
        printf(" -> %d", current);
    }
}

// Langsung ke request terkecil
int first_left = -1;
for (int i = 0; i < n; i++) {
    if (sorted[i] < initial_head) {
        first_left = i;
        break;
    }
}

if (first_left != -1) {
    total += abs(current - sorted[first_left]);
    current = sorted[first_left];
    printf(" -> %d (jump)", current);

    // Layani request sisanya
    for (int i = first_left + 1; i < n; i++) {
        if (sorted[i] < initial_head) {
            total += abs(sorted[i] - current);
            current = sorted[i];
            printf(" -> %d", current);
        }
    }
}

printf("\nTotal head movement: %d cylinders\n", total);
}

int main() {
    int requests[MAX_REQUESTS];
    int n, initial_head, disk_size;

    printf("=====\n");
    printf("  SIMULASI ALGORITMA PENJADWALAN DISK HDD\n");
    printf("=====\n\n");

    printf("Masukkan jumlah request: ");
    scanf("%d", &n);

    printf("Masukkan request queue (pisahkan dengan spasi): ");
    for (int i = 0; i < n; i++) {
        scanf("%d", &requests[i]);
    }

    printf("Masukkan posisi awal head: ");
    scanf("%d", &initial_head);

    printf("Masukkan ukuran disk (jumlah cylinder): ");
    scanf("%d", &disk_size);

```

```

printf("\n=====\\n");
printf("Request Queue: ");
for (int i = 0; i < n; i++) {
    printf("%d ", requests[i]);
}
printf("\\nInitial Head Position: %d\\n", initial_head);
printf("Disk Size: %d cylinders\\n", disk_size);
printf("=====\\n");

// Jalankan semua algoritma
fcfs(requests, n, initial_head);
sstf(requests, n, initial_head);
scan(requests, n, initial_head, disk_size, 1); // Direction: right
c_scan(requests, n, initial_head, disk_size);
look(requests, n, initial_head, 1); // Direction: right
c_look(requests, n, initial_head);

printf("\\n=====\\n");
printf("          PERBANDINGAN ALGORITMA\\n");
printf("=====\\n");
printf("Algoritma dengan total head movement terkecil\\n");
printf("adalah yang paling efisien untuk kasus ini.\\n");
printf("=====\\n\\n");

return 0;
}

```

Langkah 3: Compile program

```
gcc -o disk_scheduling disk_scheduling.c -lm
```

Langkah 4: Jalankan program dengan contoh input

```
./disk_scheduling
```

Contoh Input:

```

Jumlah request: 8
Request queue: 98 183 37 122 14 124 65 67
Posisi awal head: 53
Ukuran disk: 200

```

Topik 3: NVM Scheduling & Wear Leveling

Tujuan

Memahami karakteristik NVM (SSD) dan simulasi wear leveling.

Langkah-Langkah Praktikum

3.1 Melihat Informasi SSD

Langkah 1: Cek apakah sistem menggunakan SSD

```
lsblk -d -o NAME,ROTA
```

Penjelasan:

- ROTA = 1: HDD (rotating disk)
- ROTA = 0: SSD (non-rotating)

Langkah 2: Melihat informasi TRIM support (untuk SSD)

```
sudo hdparm -I /dev/sda | grep TRIM
```

Langkah 3: Melihat statistik I/O

```
iostat -x 1 5
```

3.2 Simulasi Wear Leveling

Langkah 1: Membuat File Program

Buat file baru dengan nama `wear_leveling.c`:

```
nano wear_leveling.c
```

Langkah 2: Menulis Kode Program

Ketik kode berikut ke dalam file:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
#include <time.h>

#define MAX_BLOCKS 100
#define BLOCK_SIZE 4096

typedef struct {
    int block_id;
    int write_count;
    int is_used;
    char data[32];
} Block;

typedef struct {
    Block blocks[MAX_BLOCKS];
    int total_blocks;
    int total_writes;
    int min_writes;
    int max_writes;
} WearLevelingSystem;

void init_system(WearLevelingSystem *sys, int num_blocks) {
    sys->total_blocks = num_blocks;
    sys->total_writes = 0;
    sys->min_writes = 0;
    sys->max_writes = 0;

    for (int i = 0; i < num_blocks; i++) {
        sys->blocks[i].block_id = i;
        sys->blocks[i].write_count = 0;
        sys->blocks[i].is_used = 0;
        memset(sys->blocks[i].data, 0, sizeof(sys->blocks[i].data));
    }
}

int find_least_worn_block(WearLevelingSystem *sys) {
    int min_idx = 0;
    int min_writes = sys->blocks[0].write_count;

    for (int i = 1; i < sys->total_blocks; i++) {
        if (sys->blocks[i].write_count < min_writes) {
            min_writes = sys->blocks[i].write_count;
            min_idx = i;
        }
    }

    return min_idx;
}

void update_stats(WearLevelingSystem *sys) {
    sys->min_writes = sys->blocks[0].write_count;
    sys->max_writes = sys->blocks[0].write_count;

    for (int i = 1; i < sys->total_blocks; i++) {
        if (sys->blocks[i].write_count < sys->min_writes) {
```

```

        sys->min_writes = sys->blocks[i].write_count;
    }
    if (sys->blocks[i].write_count > sys->max_writes) {
        sys->max_writes = sys->blocks[i].write_count;
    }
}

void write_data(WearLevelingSystem *sys, const char *data) {
    int block_idx = find_least_worn_block(sys);

    strncpy(sys->blocks[block_idx].data, data, sizeof(sys->blocks[block_idx].data) - 1);
    sys->blocks[block_idx].write_count++;
    sys->blocks[block_idx].is_used = 1;
    sys->total_writes++;

    update_stats(sys);

    printf("✓ Data written to Block %d (Write Count: %d)\n",
        block_idx, sys->blocks[block_idx].write_count);
}

void print_system_status(WearLevelingSystem *sys) {
    printf("\n");
    printf("    WEAR LEVELING SYSTEM STATUS\n");
    printf(".\n");
    printf("Total Blocks      : %d\n", sys->total_blocks);
    printf("Total Writes      : %d\n", sys->total_writes);
    printf("Min Write Count   : %d\n", sys->min_writes);
    printf("Max Write Count   : %d\n", sys->max_writes);
    printf("Wear Difference   : %d\n", sys->max_writes - sys->min_writes);

    double avg_writes = (double)sys->total_writes / sys->total_blocks;
    printf("Average Writes    : %.2f\n", avg_writes);

    printf("\n");
}

void print_block_distribution(WearLevelingSystem *sys) {
    printf("\n");
    printf("    BLOCK WRITE DISTRIBUTION\n");
    printf("\n");

    for (int i = 0; i < sys->total_blocks; i++) {
        printf("Block %2d: [%3d writes] ", i, sys->blocks[i].write_count);

        // Visual bar
        int bar_length = sys->blocks[i].write_count;
        if (bar_length > 50) bar_length = 50;

        for (int j = 0; j < bar_length; j++) {
            printf("█");
        }
    }
}

```

```

        if (sys->blocks[i].write_count == sys->max_writes) {
            printf(" △ MAX");
        } else if (sys->blocks[i].write_count == sys->min_writes) {
            printf(" ✓ MIN");
        }

        printf("\n");
    }
    printf("\n");
}

void simulate_wear_leveling(int num_blocks, int num_writes, int
show_detail) {
    WearLevelingSystem sys;
    init_system(&sys, num_blocks);

    printf("\n");
    printf("WEAR LEVELING SIMULATION STARTED\n");
    printf("\n");
    printf("Blocks: %d | Writes: %d\n\n", num_blocks, num_writes);

    srand(time(NULL));

    for (int i = 0; i < num_writes; i++) {
        char data[32];
        snprintf(data, sizeof(data), "DATA_%d", i);

        if (show_detail) {
            printf("[Write %3d/%3d] ", i + 1, num_writes);
        }

        write_data(&sys, data);

        if (show_detail && (i + 1) % 10 == 0) {
            printf("\n");
        }
    }

    printf("\n");
    print_system_status(&sys);
    print_block_distribution(&sys);

    // Analisis efektivitas
    int wear_diff = sys.max_writes - sys.min_writes;
    printf("\n");
    printf("WEAR LEVELING ANALYSIS\n");
    printf("\n");

    if (wear_diff <= num_writes / num_blocks) {
        printf("✓ EXCELLENT: Wear leveling is working effectively!\n");
        printf("  All blocks are being used evenly.\n");
    } else if (wear_diff <= num_writes / num_blocks * 2) {
        printf("⚡ GOOD: Wear leveling is working reasonably well.\n");
    }
}

```

```

    } else {
        printf("\n⚠ WARNING: Uneven wear distribution detected!\n");
    }

    printf("\nWear Efficiency: %.1f%%\n",
        (1.0 - (double)wear_diff / sys.max_writes) * 100);
    printf("===== \n\n");
}

void print_usage(const char *prog_name) {
    printf("Usage:\n");
    printf(" %s [blocks] [writes] [detailed]\n\n", prog_name);
    printf("Arguments:\n");
    printf("  blocks    : Number of blocks (default: 10)\n");
    printf("  writes    : Number of write operations (default: 100)\n");
    printf("  detailed  : Show detailed output (1=yes, 0=no, default: 0)\n\n");
    printf("Examples:\n");
    printf(" %s          # Simulate with 10 blocks, 100 writes\n",
prog_name);
    printf(" %s 20 200    # 20 blocks, 200 writes\n", prog_name);
    printf(" %s 10 100 1   # With detailed output\n\n", prog_name);
}

int main(int argc, char *argv[]) {
    int num_blocks = 10;
    int num_writes = 100;
    int show_detail = 0;

    if (argc > 1) {
        if (strcmp(argv[1], "-h") == 0 || strcmp(argv[1], "--help") == 0)
        {
            print_usage(argv[0]);
            return 0;
        }
        num_blocks = atoi(argv[1]);
    }
    if (argc > 2) {
        num_writes = atoi(argv[2]);
    }
    if (argc > 3) {
        show_detail = atoi(argv[3]);
    }

    if (num_blocks <= 0 || num_blocks > MAX_BLOCKS || num_writes <= 0) {
        printf("Error: Invalid parameters!\n");
        printf("Blocks must be between 1 and %d\n", MAX_BLOCKS);
        printf("Writes must be positive\n\n");
        print_usage(argv[0]);
        return 1;
    }

    simulate_wear_leveling(num_blocks, num_writes, show_detail);
}

```

```
    return 0;  
}
```

Simpan file dengan menekan **Ctrl+O**, lalu **Enter**, kemudian keluar dengan **Ctrl+X**.

Langkah 3: Kompilasi Program

Compile program menggunakan gcc:

```
gcc -o wear_leveling wear_leveling.c
```

Periksa apakah file executable telah dibuat:

```
ls -lh wear_leveling
```

Langkah 4: Menjalankan Program (Mode Dasar)

Jalankan program dengan parameter default (10 blok, 100 penulisan):

```
./wear_leveling
```

Output yang diharapkan:

- Status sistem wear leveling
- Distribusi penulisan per blok
- Analisis efektivitas wear leveling

Langkah 5: Menjalankan dengan Output Detail

Jalankan program dengan output detail untuk melihat setiap operasi penulisan:

```
./wear_leveling 10 100 1
```

Penjelasan parameter:

- **10** = jumlah blok
 - **100** = jumlah operasi penulisan
 - **1** = tampilkan output detail
-

Langkah 6: Eksperimen dengan Parameter Berbeda

Eksperimen 1: Meningkatkan jumlah blok

```
./wear_leveling 20 200
```

Eksperimen 2: Meningkatkan jumlah penulisan

```
./wear_leveling 15 300
```

Eksperimen 3: Rasio blok vs penulisan yang berbeda

```
./wear_leveling 5 100  
./wear_leveling 50 100
```

Eksperimen 4: Mode detail dengan banyak blok

```
./wear_leveling 20 200 1
```

Referensi Tambahan

1. Operating System Concepts (Silberschatz, Galvin, Gagne)
2. Modern Operating Systems (Andrew S. Tanenbaum)
3. Linux man pages: `man mdadm`, `man mkfs`, `man mount`
4. Linux documentation: `/usr/share/doc/`

Troubleshooting

Masalah umum dan solusi:

1. Loop device tidak bisa dibuat

```
sudo modprobe loop
```

2. Permission denied saat akses /dev

```
# Gunakan sudo untuk operasi yang memerlukan root
```

3. RAID tidak bisa dibuat

```
# Pastikan mdadm terinstall  
sudo apt install mdadm
```

4. Disk penuh

```
# Clean up file image yang tidak terpakai  
rm ~/praktikum_storage_mgmt/*.img
```

5. Program C tidak compile

```
# Install build tools  
sudo apt install build-essential
```
